

Resumer: Agentic Resume Builder & Intelligent Job Scraping Pipeline

Abstract

Resumer is a GPT-first, agentic platform designed to automate the process of job discovery, description extraction, and ATS-optimized resume tailoring. By combining stealth web scraping (Indeed and LinkedIn), NLP-driven skill enrichment, and a multi-agent CrewAI orchestration system, Resumer automatically parses candidate profiles and builds high-quality, single-page PDF resumes tailored to specific target roles. This report details the architecture, design choices, database schema, and operational workflows of the Resumer system.

Technical Project Report

1. Introduction & Project Motivation

Job hunting in the modern era is highly competitive, dominated by Applicant Tracking Systems (ATS) that filter resumes based on keyword matching. Tailoring a resume manually for every application is tedious and error-prone.

Resumer solves this challenge by providing a local-first backend and a user-friendly frontend that:

1. Bypasses scraping bot-detection to aggregate relevant job opportunities.
2. Extracts core skill requirements using lightweight NLP.
3. Automatically writes and formats resumes that align with the scraped requirements.
4. Generates a beautifully formatted PDF that is mathematically guaranteed to fit on a single A4 page.

2. System Architecture & Core Workflows

Prepared by: Antigravity (AI Assistant)

The Resumer system is structured into four main pipeline phases:

June 2026

```
[Indeed/LinkedIn Search] ---> [Stealthy Session Scraper] ---> [SQLite Database]
                                   |
                                   v
[Mistral NLP Enrichment] <--- [Extract Skills & Salary]
                                   |
                                   v
[CrewAI Multi-Agent Crew] ---> [Tailored Resume Draft (JSON)]
```

2.1 Phase 1: Search Crawl (Discovery)

The scraper executes localized, paginated searches on Indeed or LinkedIn based on keywords, locations, and filters (such as experience levels, salary, or date posted). This search returns a basic list of job listings, containing references like the Indeed Job Key (`jk`). Listings are not committed to the database immediately to keep it clean.

2.2 Phase 2: Deep Fetch (Stealth Scraping)

Using the job IDs gathered in Phase 1, the scraper makes direct HTTP requests to individual `viewjob` pages.

- **Cloudflare Bypass:** To avoid bot detection, Resumer uses `scrapling.fetchers.StealthySession` which simulates human-like TLS signatures, headers, and request pacing.
- **Direct Link Extraction:** If a user submits a single job URL directly, the system skips Phase 1, creates a mock job container, and crawls the details.
- **Persistence:** Once successfully fetched, the job details are committed to the local SQLite database.

2.3 Phase 3: NLP Enrichment

Once a job is saved, its raw description is passed to a Mistral AI NLP pipeline. Mistral parses the text to:

- Separate required hard technical skills from soft skills and generic boilerplate.
- Extract salary and compensation cues, converting them into structured records.
- Standardize metadata, which is stored in the `scraped_jobs` table.

2.4 Phase 4: Agentic Resume Generation (CrewAI Crew)

Resumer employs a multi-agent system built on CrewAI to perform the resume tailoring:

- **Job Description Analyst:** Reads and cleans messy job descriptions, converting them to clean, ATS-focused JSON.
- **Summary & Skills Writer:** Tailors the professional summary, skills, and extracurriculars to maximize ATS coverage while preserving truth.
- **Projects Tailoring Specialist:** Selects and reframes candidate projects to align with the role, adding concrete metrics.

- **Professional Work Experience Architect:** Tailors work experience bullets with action verbs and quantifiable results.
 - **Expert Resume Writer:** Integrates all sections into a cohesive resume draft that complies with single-page layout bounds.
-

3. Database Schema

Resumer uses a local SQLite database (`resumer.db`) managed via `local_backend.py` . Below are the primary tables:

3.1 `users`

Stores user profile credentials and metadata.

- `id` (TEXT, PK): Unique user identifier.
- `display_name` (TEXT): The display name of the user.
- `created_at` / `updated_at` (TEXT): Timestamps.

3.2 `profiles`

Holds the user's master resume data in JSON format.

- `user_id` (TEXT, PK, FK -> users): Owner ID.
- `truth_json` (TEXT): JSON representation of the master resume (containing all education, skills, projects, and experiences).
- `preferences_json` (TEXT): User styling and content preferences.

3.3 `projects`

Tracks individual resume tailoring tasks.

- `id` (TEXT, PK): Unique project identifier.
- `user_id` (TEXT, FK -> users): Owner ID.
- `name` (TEXT): Project name.
- `status` (TEXT): Status of generation (`running` , `completed` , `failed`).
- `job_description` (TEXT): The target job description text.

3.4 `scraped_jobs`

Aggregates scraped job descriptions, metadata, and NLP analyses.

- `id` (TEXT, PK): Indeed/LinkedIn job identifier.

- `title` / `company` / `location` / `link` / `pay` / `posted_date` (TEXT): Metadata.
 - `description` (TEXT): Raw job description.
 - `technical_skills` (TEXT): NLP-extracted technical skills.
 - `analysis_required_skills` / `analysis_preferred_skills` (TEXT): JSON lists of skills.
 - `status` (TEXT): State of scraping (`basic` , `deep_fetched` , `enriched`).
-

4. Key Implementation Highlights

4.1 Playwright PDF Generation & Auto-Fitting

To generate a professional PDF, Resumer compiles the Markdown resume template into HTML, applies custom CSS, and renders it using a headless Playwright Chromium instance.

To ensure the resume fits onto **exactly one A4 page**, Resumer implements a binary search auto-fit algorithm in `makepdf.py`:

1. **Font Size Tuning:** It binary searches the font size from `8pt` up to `12pt` to find the largest size where the page height does not exceed `1100px` (the printable A4 limit at 96 DPI).
2. **Line Height Adjusting:** It then binary searches the line height from `1.15` to `1.8` to expand spacing and elegantly fill any remaining white space.
3. **Orphan Line Detection:** It measures individual bullet point heights (`<li>`) to identify single-word text wrapping (orphans) and reports them so they can be trimmed.

4.2 FastAPI & React Dev Setup

- **Backend:** FastAPI manages background threads for scraping and CrewAI runs, avoiding API blocks during long generations.
 - **Frontend:** A sleek React + TypeScript Vite application features an interactive split-pane job library and real-time generation logs.
-

5. Conclusion & Recommendations

Resumer demonstrates the power of combining local web scraping with multi-agent orchestration. By leveraging specialized agents for writing, editing, and checking, it produces highly tailored, professionally written resumes that score high in ATS systems.

Future Roadmap:

1. Integrate auto-apply bots that submit tailored resumes automatically.

2. Support advanced multi-page formatting using similar CSS pagination rules.
3. Enable local LLM models (e.g., Llama-3 via Ollama) to remove external API costs.